

武汉大学计算机学院

本科生课程实验报告

ToyC 到 RISC-V32 的优化编译器设计与实现

专业名称： 计算机科学与技术

课程名称： 编译系统实践

指导教师： 袁梦霆

学生姓名： 万祥宇、陈伟康、黄昱熹

二〇二六年

郑 重 声 明

本团队呈交的设计报告，是在指导老师的指导下，独立进行实验工作所取得的成果，所有数据、图片资料真实可靠。除文中已经注明引用的内容外，本设计报告不包含他人享有著作权的内容。对本设计报告做出贡献的其他个人和集体，均已在文中以明确的方式标明。本设计报告的知识产权归属于培养单位。

本人签名：万祥宇、陈伟康、黄昱熹

日期：2026 年 8 月 17 日

摘 要

本文分析采用 OCaml 实现的 ToyC 优化编译器。编译器从标准输入读取源程序，经 ocamllex、Menhir、语义检查和 CFG/TAC 构建后，根据第一个用户参数决定是否进入优化总管线；`-opt` 和默认允许的裸 `opt` 均可请求优化。优化管线将 SSA 作为主要优化表示：先进行闭世界函数可达性裁剪，再执行 SSA 构造与验证、稀疏条件常量传播、GVN、复制传播、DCE、ADCE、跳转穿透、PRE 和标量清理，并在 SSA 到 CFG 的转换中处理 Phi 边复制。

在此基础上，编译器还接入非负幂余优化、ModRef 分析、只读调用公共子表达式消除、跨过程仿射摘要，以及包含循环规范化、SCEV 证明、周期/仿射折叠、强度削弱、归纳变量处理、循环删除、全局提升、有限展开和递归变换的后内联候选链。候选版本并非无条件覆盖，而是经优化计划、合法性证明、编译工作预算、热路径成本、寄存器压力和最终汇编质量 Gate 选择。

后端以带活跃空洞的精确干涉分配为主要路径，并提供增量精确分配、调用跨越约束、spill split、栈槽压紧、常量/地址重物化、叶函数无栈帧优化、比较分支融合和除法/取模强度优化。本文以源代码中的实际调用关系为依据，说明默认生产路径、条件 Gate、显式启用功能和实现边界。

关键词：ToyC；SSA；稀疏条件常量传播；循环优化；调用 CSE；精确寄存器分配；spill split；RISC-V32

目 录

1	项目概述与编译流程	1
1.1	项目目标与语言范围	1
1.2	实际入口与参数语义	1
1.3	编译器总体处理流程	1
1.4	工程模块边界	3
2	前端、CFG 与中间表示	4
2.1	词法、语法与语义分析	4
2.2	AST、CFG/TAC 与控制流构建	4
2.3	表示层次与数据流边界	6
2.4	局部 DAG 与程序可达性	6
3	SSA 优化闭环	7
3.1	SSA 构造与验证	7
3.2	SSA 优化流水线	7
3.3	SSA 优化闭环图	8
3.4	支撑分析	8
4	SSA 主线与 CFG 优化	9
4.1	SSA 优化流水线	9
4.2	早期分析与调用公共子表达式	9
4.3	CFG 标量、循环与递归变换	9
4.4	后内联候选链	10
4.5	循环优化分类图	11

5	Low IR、精确分配与代码生成	12
5.1	指令选择与调用约定	12
5.2	低级 IR 算术与控制优化	12
5.3	精确干涉分配	13
5.4	spill split、重物化与栈帧	13
5.5	后端代码生成流水线	13
6	工程组织、候选 Gate 与实现边界	14
6.1	工程组织与候选选择	14
6.2	生产路径与条件优化	14
6.3	条件优化功能总表	15
6.4	团队 Git 工作流	15
6.5	创新点与实现边界	18
7	总结	19

1 项目概述与编译流程

1.1 项目目标与语言范围

本项目实现从 ToyC 源程序到 RISC-V32 汇编的单文件编译器。ToyC 是用于编译系统实践的 C 语言子集，支持全局/局部变量与常量、int/void 函数、块、赋值、条件、while、break、continue、return 和函数调用；数组、指针、I/O 与多文件编译不在语言范围内。工程使用 OCaml 和 Dune，前端由 ocamllex 与 Menhir 构建，后端面向 RV32IM 指令集输出汇编。

报告只描述编译器自身的功能；优化功能以实际驱动调用和优化模块为依据。

1.2 实际入口与参数语义

bin/main.ml 的 driver 组织以下调用链：

```
stdin/lexbuf → Lexer/Parser → Semant.check → Cfg.build_program  
→ Driver_options.optimization_requested → optimize (条件)  
→ Ir.lower_program → Codegen.emit → stdout
```

driver_options.ml 检查第一个用户参数：-opt 请求优化，裸 opt 默认作为兼容形式请求优化；TOYC_NO_PLAIN_OPT_ALIAS=1 可以关闭后者。无优化时，程序由 CFG/TAC 直接进入低级 IR；优化时，程序先经过闭世界可达性裁剪、SSA 和多层候选选择，再进入后端。无论是否优化，最终都经过 Ir.lower_program 与 Codegen.emit。

1.3 编译器总体处理流程

图 1.1 将普通路径、SSA 主线、程序级分析、后内联候选和后端选择放在同一张图中。优化候选并非无条件叠加：结构资格、证明、工作预算、寄存器压力和最终汇编质量共同决定候选是否物化及是否成为 winner。

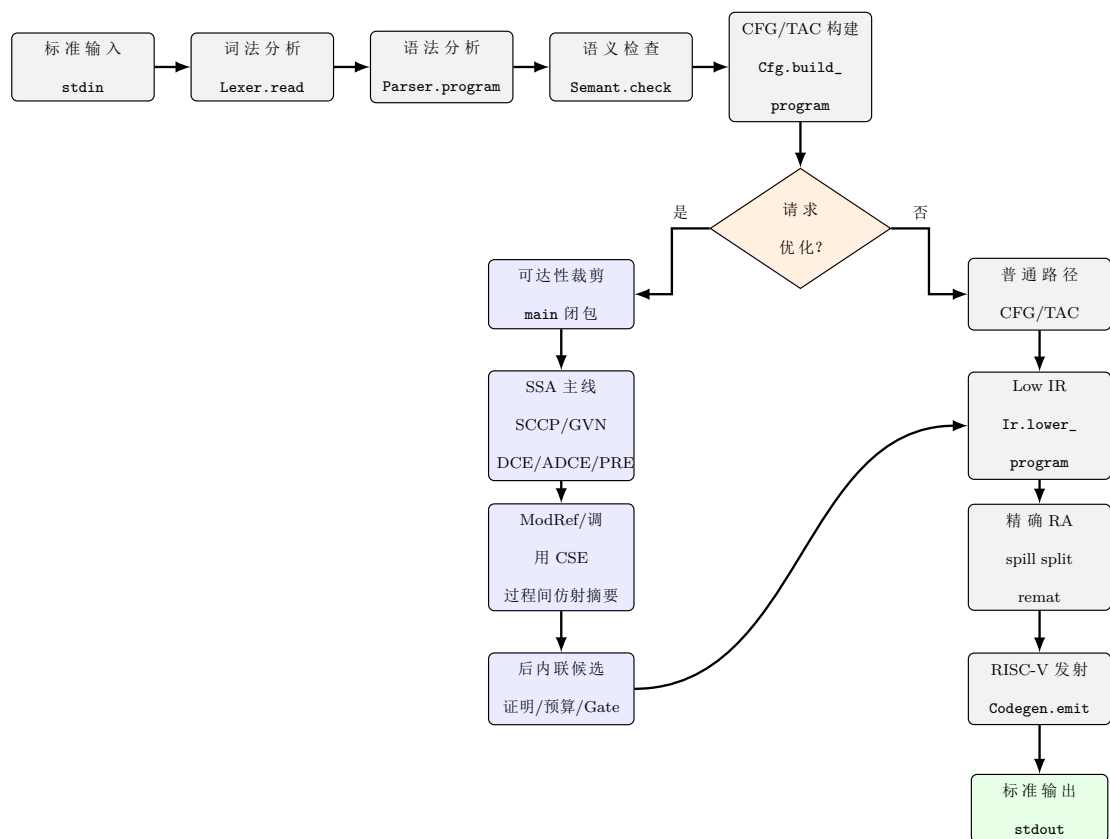


图 1.1 ToyC 编译器代码处理流程与优化 Gate

1.4 工程模块边界

`lib/dune` 将 AST、语义、CFG 分析、SSA、循环分析、优化计划、ModRef、IR、spill split、fallthrough 与 codegen 编译进库；`bin/dune` 构建 driver。模块被编译进库不等于每次运行都执行，本文以 `main.ml` 的实际调用为准区分生产路径、条件优化和支持性模块。

2 前端、CFG 与中间表示

2.1 词法、语法与语义分析

`lib/lexer.mll` 以 `read` 为入口识别关键字、C 风格标识符、十进制整数、运算符、界符、空白和注释。关键字规则位于标识符规则之前；`line_comment` 与 `block_comment` 分别消费单行和块注释，再返回 `read`。未知字符和未闭合块注释进入错误分支。

`lib/parser.mly` 由 Menhir 生成分析器，按逻辑或、逻辑与、关系、加减、乘除模、一元和基本表达式分层表达优先级，语义动作直接构造 `Ast.program`。`Semant.check` 先登记函数签名与全局声明，再检查函数体的作用域、常量赋值、参数数量、循环控制、返回形式和 `main` 入口。符号表以哈希表栈保存嵌套作用域，并记录循环深度。

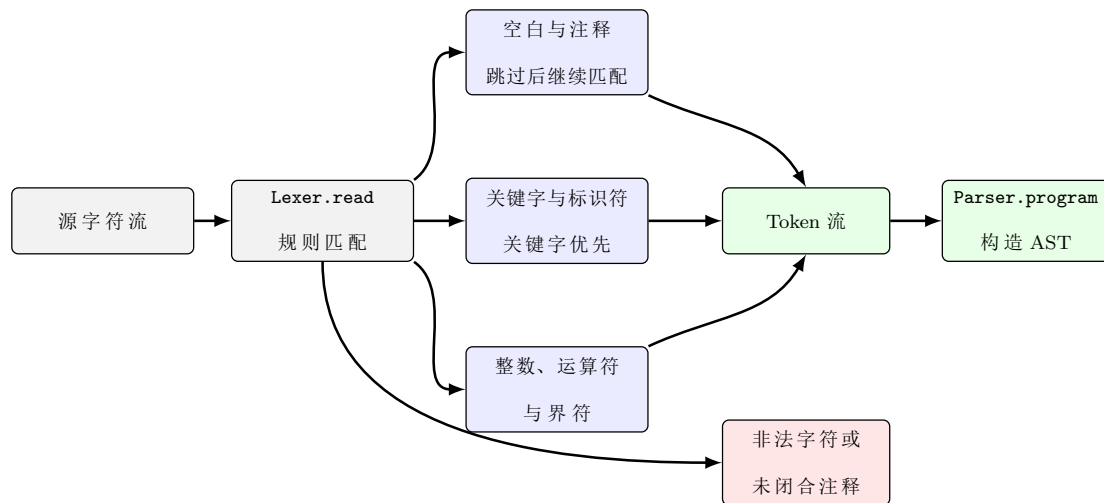


图 2.1 ToyC 词法分析器结构

2.2 AST、CFG/TAC 与控制流构建

AST 以代数数据类型表示表达式、语句、函数和顶层声明。`Cfg.build_program` 将 AST 转为函数 CFG；每个基本块由标签、TAC 指令序列和唯一终结指令组成。

表 2.1 ToyC 词法单元与识别规则

类别	规则示例	输出或处理
关键字	<code>int, void, const, if,</code> <code>else, while, break,</code> <code>continue, return</code>	输出对应保留字 Token
标识符	<code>[_a-zA-Z][_a-zA-Z0-9]*</code>	输出 ID, 关键字优先匹配
整数	十进制数字串	转换为 32 位整数常量 Token
复合运算符	<code><=, >=, ==, !=, &&, </code>	输出关系或逻辑 Token
单字符符号	算术、关系、赋值、括号和分 隔符	输出对应 Token
空白与注释	空 格/制 表/换 行、 <code>//</code> 、 <code>/*...*/</code>	消费后继续调用 <code>read</code>
非法输入	兜底字符、未闭合块注释	抛出含位置信息的错误

TAC 覆盖常量、二元/一元运算、移位、复制、地址、加载、存储和调用；终结器覆盖返回、跳转、条件分支与尾调用。

局部变量和前八个形参主要映射到虚拟寄存器，形参入口使用虚拟寄存器 10–17 对应 a0--a7；更多参数从 $fp+(i-8)*4$ 读取。全局变量通过 TLa 配合加载/存储访问。if、while、break、continue 和短路逻辑由显式基本块与控制边实现。源端 `return f(...)` 先形成普通调用加返回，递归/尾调用优化随后可将其改写为 `TTailCall`。

2.3 表示层次与数据流边界

图 2.2 表明 CFG/TAC 是贯穿中端的持久表示，SSA 是优化阶段的临时表示；`from_ssa` 完成 Phi 边复制后重新得到 CFG/TAC，后端再降低到 Low IR。

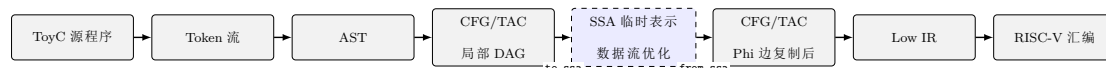


图 2.2 编译器中间表示及转换关系

2.4 局部 DAG 与程序可达性

连续纯表达式段由 `process_segments` 交给 DAG。Dag.intern 通过 hash-consing 消除局部公共子表达式；常量折叠使用 32 位回绕语义；`try_simplify` 实现恒等式、自比较和小常数代数化简；`mark_live`、`node_depth` 与 `flatten_roots` 完成根可达发射、共享子树复用和 Sethi-Ullman 风格求值次序。调用、赋值和短路逻辑被排除在纯表达式段之外。

优化入口的 `Cfg.prune_program_reachability` 以 `main` 为根，沿 `TCall`、`TCallVoid` 和 `TTailCall` 建立直接调用闭包，删除不可达函数。缺少入口、函数名冲突或 CFG 异常时保持原程序，采用 fail-closed 策略。调用 CSE 后再次裁剪，使失去调用点的函数不会进入后续分析。

3 SSA 优化闭环

3.1 SSA 构造与验证

优化入口默认对函数调用 `Ssa.optimize_ssa`。 `to_ssa` 先计算支配关系与支配边界，在变量定义的支配边界插入 `Phi`，再沿支配树为定义和使用分配版本寄存器。 `verify_ssa` 检查单定义、使用可达性、`Phi` 前驱、支配关系及版本约束；任何不能建立一致 SSA 的函数均采用保守处理。 `TOYC_NO_SSA=1` 可关闭该阶段， `TOYC_NO_SSA_CALLS=1` 只在显式要求时跳过含调用函数。

3.2 SSA 优化流水线

表 3.1 SSA 主线的优化阶段

阶段	主要函数	功能
构造与验证	<code>to_ssa</code> 、 <code>verify_ssa</code>	<code>Phi</code> 插入、版本重命名及 SSA 不 变量检查
稀疏传播	<code>ssa_sccp</code> 、分支/ <code>Phi</code> 简化	在可执行边和值格上折叠常量、 删除不可达分支
值编号	<code>ssa_gvn</code> 、 <code>ssa_copy_prop</code>	识别等价表达式并传播副本
删除	<code>ssa_dce</code> 、 <code>ssa_adce</code>	删除无用定义及由控制依赖证明 不可观察的计算
控制与表达式	<code>ssa_jump_thread</code> 、 <code>ssa_pre</code>	穿透已知跳转并消除部分冗余表 达式
收束与反构造	<code>ssa_scalar_cleanup</code> 、 <code>from_ssa</code>	清理副本/ <code>Phi</code> ，生成边复制并恢 复 CFG/TAC

优化序列会在主要变换之间重复分支/Phi 简化、复制传播和 DCE，以消费前一阶段产生的新机会。ADCE 使用控制依赖和必须终止证明保留可观察控制流；PRE 在支配关系允许时插入表达式，使部分可用表达式成为完全可用。`from_ssa` 不简单丢弃 Phi，而是按前驱边生成 Phi edge moves，并以 `parallel_copies` 处理复制环和亲和合并。

3.3 SSA 优化闭环图

图 3.1 强调 SSA 是中端优化的闭环表示：输入和输出仍是 CFG/TAC，内部每一步均受验证和清理约束。



图 3.1 SSA 优化闭环

3.4 支撑分析

SSA 主线共享 CFG 可达性、支配树、支配边界、控制依赖、活跃性和 reaching definitions。循环候选还会使用 LoopInfo、LCSSA 和 SCEV；全局内存与调用候选使用 ModRef；过程间折叠使用仿射函数摘要。分析不能证明安全性时，变换拒绝候选并保留输入 CFG。

4 SSA 主线与 CFG 优化

4.1 SSA 优化流水线

优化模式中，`Ssa.optimize_ssa` 以 CFG/TAC 为输入，依次执行 `to_ssa`、`verify_ssa`、SCCP、分支/Phi 简化、GVN、复制传播和 DCE；随后重复消费传播机会，再执行 ADCE、跳转穿透、PRE 和标量清理，最后由 `from_ssa` 通过 Phi edge moves 恢复 CFG。TOYC_NO_SSA=1 可关闭整个阶段，TOYC_NO_SSA_CALLS=1 可按函数类型跳过含调用函数。

4.2 早期分析与调用公共子表达式

SSA 主线之后，优化器尝试非负幂余变换；`Modref.analyze` 建立函数读写摘要，再由 `Cfg_call_cse.optimize_program` 对已证明 readnone 的调用执行块内公共子表达式消除。调用 CSE 后重新执行程序可达性裁剪，避免失去调用点的函数进入后续分析。`Cfg_interproc_affine.analyze` 生成过程间仿射摘要，为后续闭式折叠和候选证明提供信息；这些阶段均受相应的 TOYC_NO_* 条件控制。

4.3 CFG 标量、循环与递归变换

`Cfg.optimize_program` 的函数级基础阶段依次包括可达性清理、DCE、常量传播、复制传播、立即数折叠、复制合并、CFG LICM、标量清理、循环反转、计数循环折叠、关联递归优化、仿射递归展开和尾递归转换。随后还可能执行非循环调用内联、有界常量递归展开、树递归展开、自内联、循环自内联、残余递归版本化、函数可达性再次裁剪和热尾部 SCC 合并。

表 4.1 CFG 与递归优化

类别	代表函数/模块	具体作用
结构清理	<code>prune_unreachable</code> 、	删除不可达块和无用定义，传
	DCE、传播、合并	播常量与副本
循环基础	<code>Cfg LICM</code> 、 <code>loop_invert</code> 、	外提盈利不变量、规范循环方
	计数循环 Fold	向、折叠计数循环
递归变换	<code>tail_rec_opt</code> 、关联递归、	将可证明递归改写为迭代或
	仿射递归	闭式形式
调用变换	<code>inline_func</code> 、 <code>bounded</code>	在结构和成本允许时展开调
	<code>self inline</code> 、SCC 合并	用或合并热尾部

4.4 后内联候选链

默认候选由 `post_inline_j04` 至 `post_inline_j31m` 构成，并通过统一的父子关系、计划描述、合法性证明和后端 Gate 选择。下文在保留内部候选编号的同时，给出每个编号对应的优化含义：

- 基础标量与循环：J03R（全 SSA 链覆盖与 CFG 必常量 `div/rem`）、J04（后内联 SSA 标量清理）、J05（Loop Simplify/LCSSA 循环规范化）、J06（非陷阱标量 LICM）、J07/J07R（SCEV 证明与 Fold 统一管理）、J08（有界周期状态转移与加法闭式）、J09（仿射递推矩阵快速幂）、J10（派生归纳变量的常量乘法强度削弱）、J11（死/重复 IV 消除与精确 trip 倒计数）、J12（有限且无副作用循环删除）、J13（全局 load LICM）、J14（全局内存 SSA 提升与 store sinking）和 J15（必执行 Div/Mod 及只读调用 LICM）；
- 折叠与展开：J23A/J23AR（常量模 IV 强度削弱及共享分析穷举搜索）、J23B（精确 0-8 trip 完整循环展开与 SSA 清理）、J23C（常量无符号除法 lowering）、J24P（常量 trip 大周期加法平移摘要）、J24F（动态 trip 整周期快进并保留余数循环）、J25P（带区间/同余证明的动态 phase 快进）和 J26A（周期仿射向量轨道组合与矩阵快速幂）；其中 J24P+J23A、J24P+J23B 表示相应周期摘要与取模强度削弱或完全展开的组合候选；

- 状态与过程间：J27S（有限控制/IV 相位图与分段仿射周期幂）和 J28I（过程间仿射直接调用/全局效果摘要及重放）；
- 多出口与归约：J29E（有界精确可归约多出口克隆）、J29F（动态 unit-IV 等值多出口规范化）、J29G（一般步长/谓词及无环菱形多出口规范化）、J30R（循环不变量条件外分区与真假分支仿射矩阵幂）和 J31M（私有整数归约的 factor-4 多累加器、余数与出口合并）。

J30U（SSA 的 factor-2/4 部分展开与溢出安全的原循环余数路径）仅在环境变量 TOYC_ENABLE_PARTIAL_UNROLL=1 开启且未被禁止时追加。候选可能因结构、证明、预算、热成本、寄存器压力或最终汇编质量不满足要求而回退到父版本。

4.5 循环优化分类图

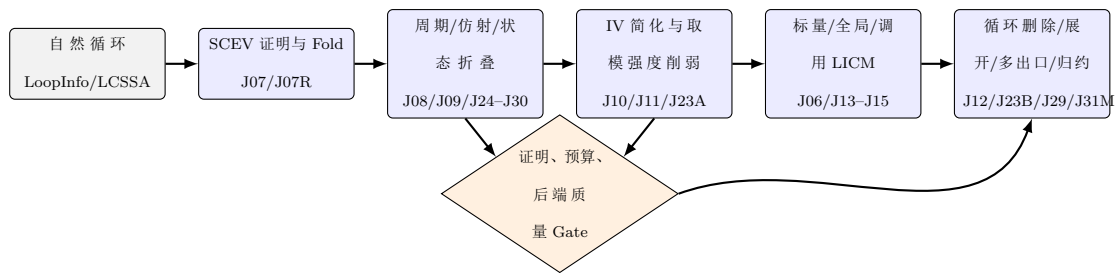


图 4.1 循环优化功能分类与 Gate 约束

该分类图中的 J 编号是候选计划的标识，不表示所有候选在每个输入上都会执行；真正改变 CFG 的候选必须通过合法性、预算和质量选择。

5 Low IR、精确分配与代码生成

5.1 指令选择与调用约定

`Ir.lower_tac` 将 TAC 映射为 RV32 指令:算术和逻辑运算映射为 `add/sub/mul/div/rem`、按位运算和移位指令;访存映射为 `lw/sw`;调用映射为 `jal/jalr`。`fused_cbr` 在条件值没有后继活跃使用时,把比较和分支融合为 `blt/bge/beq/bne` 等真实控制转移。`TTailCall` 由 `Ir.lower_tail_call` 生成尾跳, `codegen` 在跳转前执行帧拆除。

虚拟寄存器约定为: 2 是 `sp`, 8 是 `fp`, 10-17 是 `a0--a7`, 18 以上为一般虚拟寄存器。调用前八个参数放入 `a0--a7`, 其余参数写入按 16 字节对齐的 `outgoing stack area`; 返回值从 `a0` 搬回目标虚拟寄存器。`Ir.lower_program` 在候选 CFG 之间建立 `fresh-vreg floor`, 避免中间变换产生的临时寄存器与 Low IR 临时寄存器冲突。

5.2 低级 IR 算术与控制优化

最终 lowering 会尝试以下默认或受 Gate 约束的变换:

- 常量除法和取模强度削弱, 并在证明条件满足时使用零观察掩码;
- 常量乘法强度削弱, 将乘法分解为移位和加减链;
- `quotient/remainder sharing`, 避免同一除数上的重复计算;
- `compare-branch fusion`, 减少条件值物化;
- `drop_fallthrough`, 删除物理布局中冗余的下一块跳转。

低级 ADDI 合并、密集硬件取余、`branch modulo update`、`constant pool` 和部分后期选择器受明确环境开关或后端质量 Gate 控制, 属于条件功能, 不默认声称每个函数都会触发。

5.3 精确干涉分配

`analyze_allocation` 同时建立 CFG 后继、自然循环权重、逐指令活跃性、调用跨越信息、固定寄存器禁用颜色、值权重和保留活跃空洞的精确干涉关系。`allocate_precise_reference` 逐候选重算 forbidden colors; `allocate_precise_incremental` 增量维护 available、saturation 和 forbidden-color 统计; `allocate_precise` 默认选择增量路径, 显式关闭时才回退 reference。 `allocate_legacy` 作为兼容基线保留, 但不是主要分配路径。

普通 caller pool 为 `t0--t3`, 参数池为 `a0--a7`, callee pool 为 `s1--s11`。只有无参数、无调用/尾调用的 leaf 函数才可将参数寄存器并入普通 caller pool; 跨越调用的值必须满足 callee-save 约束。`verify_allocation` 检查每个值是否完成映射或 spill、固定寄存器冲突、干涉冲突及调用跨越约束。

5.4 spill split、重物化与栈帧

`select_spill_plan` 支持三类重物化配方: 整数常量、地址和简单加立即数链。`Spill_split.select` 与 `Spill_split.validate` 根据完整 CFG 活跃性选择 entry reload、edge reload、exit store/load 和 resident 值, 并压紧 stack slots、删除不必要的 spill store。单驻留策略限制每块栈值驻留数, 多驻留策略在颜色和调用约束允许时分割活跃范围。

`frame_layout` 的原始帧大小为:

$$8 + 4 * \text{num_slots} + 4 * \text{stack_spill_count} + 4 * \text{callee_save_count}$$

随后帧大小向上对齐至 16 字节。叶函数无栈帧优化可省去完全不需要的 `sp/fp/ra` 操作; 非叶函数保存实际使用的 callee-save 寄存器。大于有符号 12 位范围的帧偏移由暂存寄存器计算有效地址, 普通返回和尾跳均使用一致的 teardown。

5.5 后端代码生成流水线

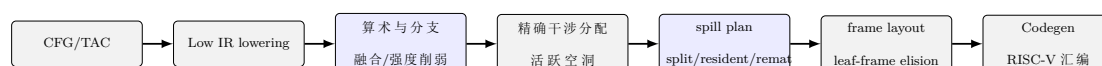


图 5.1 后端代码生成流水线

6 工程组织、候选 Gate 与实现边界

6.1 工程组织与候选选择

lib/dune 编译 CFG/SSA 分析、ModRef、仿射摘要、优化计划、IR 分配、spill split、fallthrough 和 codegen 模块;bin/main.ml 负责组织运行时管线。Optimization_plan 保存候选描述、证明条件、工作预算和 winner 选择信息，使候选能够延迟物化，并在失败时回到父版本。

6.2 生产路径与条件优化

表 6.1 生产路径与条件优化

类别	功能	进入条件
默认生产路径	可达性裁剪、SSA 主线、ModRef/调用 CSE、CFG 基础优化、默认后内联候选、精确 RA、spill split、基本 Low IR 与代码生成	优化模式请求且满足各自结构/安全 Gate
条件候选	J30U 部分展开、低级 ADDI 合并、branch modulo update、dense hardware remainder、constant pool、部分后期闭式/特化选择	环境变量显式启用或默认允许且通过后端质量 Gate

条件 Gate 的顺序通常是：结构资格检查、SSA/LoopInfo/SCEV/ModRef 证明、CFG 大小与工作预算限制、热路径成本评估、寄存器压力与 spill 评估、Low IR 和最终汇编质量比较。候选退化时回到父版本，主 CFG 不被不安全或不盈利的变

换破坏。TOYC_NO_* 和 TOYC_ENABLE_* 环境变量为编译器优化提供条件控制，例如关闭 SSA、ModRef、增量精确 RA、可达性裁剪，或打开部分展开与低级 ADDI 合并。

6.3 条件优化功能总表

表 6.2 条件启用与晚期选择的编译器优化

层次	优化家族	功能与条件
后内联终端选择	循环旋转、残余热循环、非零 Div/Rem LICM	在候选链后重试可证明且后端质量改进的循环形态
J32S 系列（分支/Phi 简化、SSA ADCE、跳转穿透、PRE 与标量 CFG 清理）及 J33U（通用循环 unswitch）	分支/Phi 简化、激进死代码消除、SSA jump threading、部分冗余消除、标量 CFG 清理与通用循环 unswitch	由结构、支配关系和值域证明生成候选，并接受工作预算和质量 Gate
晚期 Fold	certified scalar/chained/global chained Fold、post-range mod-IV、 post-fold cleanup	在前序候选暴露新闭式机会后再次证明并选择
程序级晚期选择	leaf inline、热常量特化、全局提升、条 件链融合、区域重定向	对最终 CFG 重新构造候选，要求代码规模、调用图、内存效果和后端成本不退化
显式 opt-in	J30U（2/4 倍部分展开）、J106S（循环 小常量外提）、J48S（single-use code sink）、J68A（single-use Addi 链组 合）、J69B（branch-mod 更新）、J80R （硬件 rem 识别）、J47L（常量池提取）	仅在相应 TOYC_ENABLE_* 开关满足时尝试，并继续通过合法性与后端 Gate

6.4 团队 Git 工作流

团队采用基于 Git 的集中式协作流程：成员在各自分支上完成编译器模块的修改，通过具有明确类型的提交记录保存阶段性成果，再将经过检查的修改合并到主开发线上。图 6.1 展示了当前仓库中的提交演进和分支汇合关系；其中提交消息按照功能、修复、构建和初始化等类别组织，便于追踪 CFG、SSA、IR、Codegen 及优化模块的演进。



图 6.1 团队 Git 提交与协作 workflow

团队分工如下：

表 6.3 团队成员分工

成员	负责内容
万祥宇	承担了项目测试工具集 (<code>testkit/</code>) 中测试用例的编写工作, 负责设计、编写功能与性能测试用例, 覆盖编译器的各个模块和功能。包括, 正确性验证: 测试用例通过计算运行时间与 <code>gcc-02</code> 编译的结果进行验证, 性能压力设计: 包含 stress 循环 (如 <code>while (i < 600000000)</code>), 确保编译器生成的代码有足够的执行时间。优化覆盖设计: 针对每个 pass 设计测试, 包括: SSA SCCP GVN LICM 函数内联后的 SSA, 边界测试: 包含嵌套控制流、深度调用链等边界场景。
陈伟康	负责 ToyC 编译器的整体路线规划, 并实现其中后端及优化。规划的管线为 <code>AST → CFG (转 DAG 局部优化) → SSA → CFG 全局优化 → 内联后二次 SSA 循环优化 → Low IR → 寄存器分配 → RISC-V 汇编</code> 。实现上依次完成: CFG 构建及将基本块转 DAG 的局部优化 (常量折叠、CSE 等); SSA 管线 (SCCP、GVN、DCE 等), 结束后经 phi-aware 反构造转回 CFG; CFG 全局优化 (cleanup、LICM、loop inversion、基于 SCEV 的递推消除 Fold、尾递归优化、内联等); 内联后再次建立 SSA 进行循环优化 (Loop Simplify/LCSSA、SCEV 分析、IV 强度削减、完全展开、全局内存优化等); 常量除法/取模/乘法强度削减与 IR Lowering; 以及后端: RISC-V 32 汇编代码生成与栈帧管理、图着色寄存器分配 (加权分配、coalescing 等)、spill 优化 (重物化、活跃区间分裂等)、叶函数 frame/ra 消除、fall-through 规范化等。

表 6.3 团队成员分工（续）

成员	负责内容
黄昱熹	负责编译器前端功能实现、前端模块与编译管线衔接、功能与性能测试协作以及报告撰写。实现 ToyC 词法规则、Menhir 语法规则、AST 数据结构、嵌套作用域符号表与语义检查；完成变量/常量声明、函数调用、参数匹配、循环控制、返回语句、短路逻辑和编译期常量表达式从源代码到 CFG/TAC 的功能路径。完成前端模块与后续优化、后端模块的接口衔接，处理前端生成中间表示与下游模块之间的数据传递和错误反馈；协作进行功能与性能测试，补充典型输入、边界场景和异常语义检查，并完成报告撰写。

6.5 创新点与实现边界

当前实现的主要工程特点包括：

1. 以 SSA 验证、SCCP/GVN/ADCE/PRE 和 Phi edge moves 组成可恢复的 SSA 主线；
2. 以证明、工作预算和最终后端质量 Gate 管理后内联候选，支持多版本探索和父版本回退；
3. 将 ModRef 调用 CSE、跨过程仿射摘要和闭世界函数裁剪纳入程序级优化；
4. 将精确干涉空洞、增量禁色维护、spill split、重物化、栈槽压紧和叶帧消除联合到后端选择。

实现边界包括：候选依赖输入结构和证明，不同输入不一定触发同一版本；`allocate_legacy` 保留作分配回退路径；部分低级优化需要显式 `opt-in`；静态成本模型决定候选选择；闭世界裁剪依赖 ToyC 函数不可作为值且程序入口唯一为 `main` 的语言约束。

7 总结

当前 ToyC 编译器形成了分层、可证明、可选择的优化编译系统。前端由 ocamllex、Menhir、AST、符号表和语义检查组成；CFG 阶段保留 DAG 局部优化、短路控制流、调用和尾调用表示；优化模式以 `main` 的闭世界可达性为边界，默认接入 SSA 构造验证、SCCP、GVN、复制传播、DCE、ADCE、跳转穿透和 PRE。

SSA 主线之后，ModRef 调用 CSE、跨过程仿射摘要、循环规范化、SCEV 证明、周期/仿射折叠、强度削弱、归纳变量优化、循环删除、全局提升、递归变换和有限展开通过候选计划组织。后内联阶段维护具有父子关系的候选版本，候选经过合法性、预算、热成本、寄存器压力和最终汇编质量比较后选择 winner；因此优化器同时具备多版本探索和失败回退能力。

后端以 Low IR 为边界，支持比较分支融合、除法/乘法强度削弱、商余共享和 fall-through 规范化；分配器主要使用带活跃空洞的精确干涉分配，并配合调用跨越约束、增量禁色维护、spill split、重物化、栈槽压紧和叶函数无栈帧。最终由 Codegen 统一处理正常返回与尾调用 teardown，输出 RV32 汇编。

本设计文档以当前源码的实际调用点为准，区分默认生产路径、条件 Gate、显式 opt-in 功能和实现边界；重点说明优化器的表示转换、证明机制、候选选择、寄存器分配与汇编生成过程。

教师评语评分

评语:

评分:

评阅人:

年 月 日

(备注:对该实验报告给予优点和不足的评价,并给出百分制评分。)